

Trabalho Prático 1

Agentes conversacionais de busca

1. Introdução

O problema proposto foi a implementação de um agente conversacional, utilizando a biblioteca smolagents, que seja capaz de interagir com o mundo a partir de um grande modelo de linguagem (*Large Language Model - LLM*). O agente apresentado em questão tem como objetivo informar rotas entre diferentes pontos de Belo Horizonte, tendo como seu alvo principal os diversos museus da cidade. Para isso, foi desenvolvido um código em python utilizando as ferramentas recomendadas pelo enunciado para geração do grafo, configuração do ambiente, edição do prompt passado ao agente e exibição do resultado.

Esta documentação divide-se nos seguintes módulos:

2. Motivação: Descrição do problema a ser resolvido e visão pessoal da utilidade do agente.
3. Metodologia: Apresenta informações técnicas sobre o problema trabalhado e soluções adotadas, além de detalhar a arquitetura utilizada para integrar o agente com os algoritmos de busca escolhidos.
4. Testes realizados e resultados obtidos: Apresenta alguns exemplos de entradas e de saídas.
5. Conclusões e comentários sobre dificuldades.

2. Motivação

Ao se viajar para uma nova cidade é comum buscar por locais turísticos e culturais para se visitar. Entre os principais destinos procurados por turistas, museus se destacam nas escolhas. De acordo com pesquisa divulgada pelo portal Diário do Comércio, o Centro Cultural do Banco do Brasil (CCBB) de Belo Horizonte está entre os 50 museus mais visitados no mundo, em 41º no ranking. Além disso, estudos apontam que diversas pessoas só vão a museus quando viajam [GOSLING et al. 2014] e que 11% dos turistas estrangeiros que vêm ao país buscam o turismo cultural, de acordo com o ministério do turismo. Dessa forma, urge a necessidade de um sistema que trace rotas entre os mesmos de forma simples e amigável.

3. Metodologia

O problema a ser trabalhado neste trabalho prático é o de rotas para museus na cidade de Belo Horizonte em Minas Gerais, Brasil. A partir da biblioteca osmnx em python foi importado o grafo da cidade e realizado o filtro de só considerar as ruas principais (*custom_filter='["highway"~"primary|secondary|tertiary"]*) e posteriormente de acordo com as restrições de prédio como museu (*"tourism": "museum"*).

Para evitar o problema de conexão com servidor tratado no apêndice A, uma vez carregado o grafo da cidade foi salvo em um arquivo local denominado "bh.graphml". O grafo da cidade com esses filtros apresenta 3834 nós e 11668 arestas, tendo como objetivo encontrar rotas entre os 14 museus filtrados, apresentados na Figura 1 abaixo.

```

Node 27591003: Casa Kubitschek
Node 29095351: Museu de História Natural da UFMG
Node 29096783: Museu Mineiro
Node 32448895: Centro de Arte Contemporânea e Fotografia
Node 32600635: Museu de Artes e Ofícios
Node 35968129: Espaço do Conhecimento UFMG
Node 36417185: Museu da Imagem e do Som
Node 351980159: Museu de Ciências Morfológicas da UFMG
Node 1034410884: Museu de Arte da Pampulha
Node 2285761096: Museu Brasileiro do Futebol
Node 2460331365: Casarão histórico do Museu Histórico Abílio Barreto
Node 2823629282: Edifício Sede MHAB
Node 5395802458: Memorial Minas Gerais Vale
Node 8201639218: Museu de Ciências Naturais - Prédio 40

```

Figura 1 - Museus de Belo Horizonte retornados com os filtros

Para busca de rotas sem informação foram utilizados os algoritmos de Custo Uniforme/Dijkstra e Busca em profundidade (*Depth-First Search, DFS*), sendo a primeira a partir da biblioteca *networkx* e a última implementada de forma manual. Por outro lado, para a busca com informação, foi utilizado o algoritmo A* para avaliação do caminho possível, utilizando o método `nx.astar_path()`.

Inicialmente houve a tentativa de utilizar o modelo `ollama/qwen2:7b`, recomendado no Onboarding do curso de agentes da Hugging Face, entretanto o mesmo se mostrou ineficiente e demorado na máquina testada. Por fim foi utilizado o modelo `ollama/qwen3:8b` em outra máquina, decisão detalhada no apêndice A.

3.1. Arquitetura utilizada para integrar agentes e busca

Para integrar o agente desenvolvido com os algoritmos de busca, as funções de busca foram implementadas como ferramentas (*tools*) a serem utilizadas pelo agente. Cada teste foi realizado deixando claro qual tipo de busca seria realizada no momento, sem haver conflito ou decisão relacionada por parte do agente. Um esquema visual está apresentado abaixo na Figura 2.

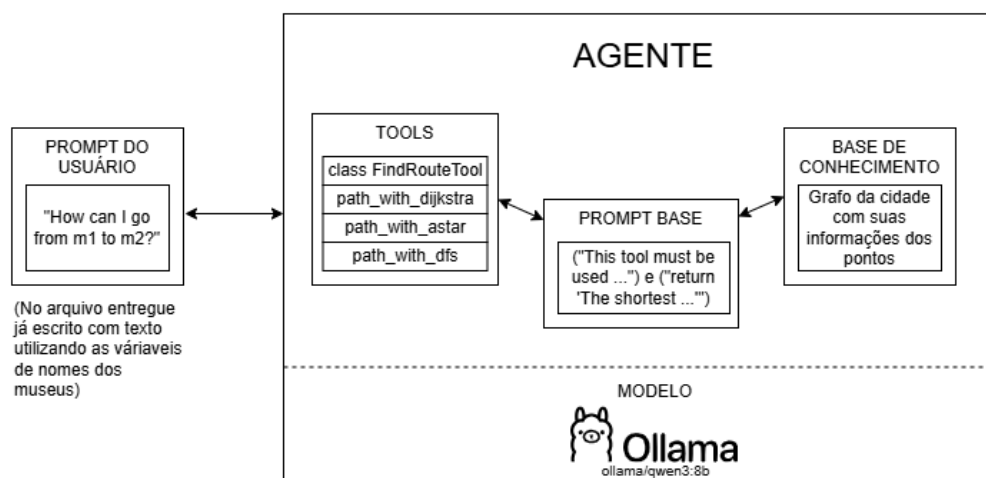


Figura 2 - Arquitetura utilizada para integrar agente com busca

Além disso, todo o código foi desenvolvido em um notebook python (.ipynb) e está dividido em seções devidamente comentadas com as células de texto em markdown. As seções apresentadas são: (0) Título e informações sobre o trabalho; (1) Configurações iniciais e impressão de dados sobre o grafo obtido; (2) Algoritmos de busca; (3) Exemplos de conexão do agente a partir do código e de LLM; (4) Definições necessárias para o agente; (5) Execução do agente e (6) Comparação e análise dos resultados.

4. Testes realizados e resultados obtidos

Durante a implementação foram realizados diversos testes e um subconjunto de prompts analisados está apresentado abaixo nesta documentação simplificada. Os prompts de entrada apresentados para os diferentes algoritmos de busca seguem a ordem de origens e destinos apresentados como

- A) Museu de História Natural da UFMG → Museu de Ciências Morfológicas da UFMG
- B) Centro de Arte Contemporânea e Fotografia → Museu da Imagem e do Som
- C) Museu de Ciências Morfológicas da UFMG → Museu de Arte da Pampulha

As saídas foram salvas em arquivos de texto denominados “resposta_agente_dfs.txt” e “resposta_agente_astar”, com um maior detalhamento das mesmas abaixo.

4.1. Quantidade de nós nas rotas retornadas pelo agente

Para o teste com busca sem informação utilizando a estratégia de busca em profundidade, os três testes supracitados realizados resultaram em rotas com 706, 165 e 51 nós, respectivamente. Por outro lado, os testes de busca informada com o algoritmo A*, para os mesmos museus de origem e destino, retornou como resultado caminhos com 38, 9 e 16 pontos, respectivamente. A partir desse resultado intermediário já é possível perceber a grande diferença entre as duas estratégias, entretanto, outros testes mais simples não utilizando o agente também foram realizados para melhor clareza de resultados, conforme detalhado abaixo.

4.2. Comparação entre os algoritmos de busca

Na seção 6 do arquivo notebook python interativo (*Interactive Python Notebook*, “.ipynb”) entregue é apresentada uma comparação entre os diferentes algoritmos de busca como exemplo, utilizando como origem e destino os museus Museu de História Natural da UFMG e Casa Kubitschek. Na célula em questão, não é utilizado agente para essa comparação visando simplificação e rapidez ao executar. Sua saída está exemplificada na Figura 3 abaixo.

```
Rota com Dijkstra: [29095351, 9048339266, 303343655, 1494808236, 1494808222, 6429380808, 587112583]
Quantidade de nós: 48

Rota com A*: [29095351, 9048339266, 303343655, 1494808236, 1494808222, 6429380808, 587112583]
Quantidade de nós: 48

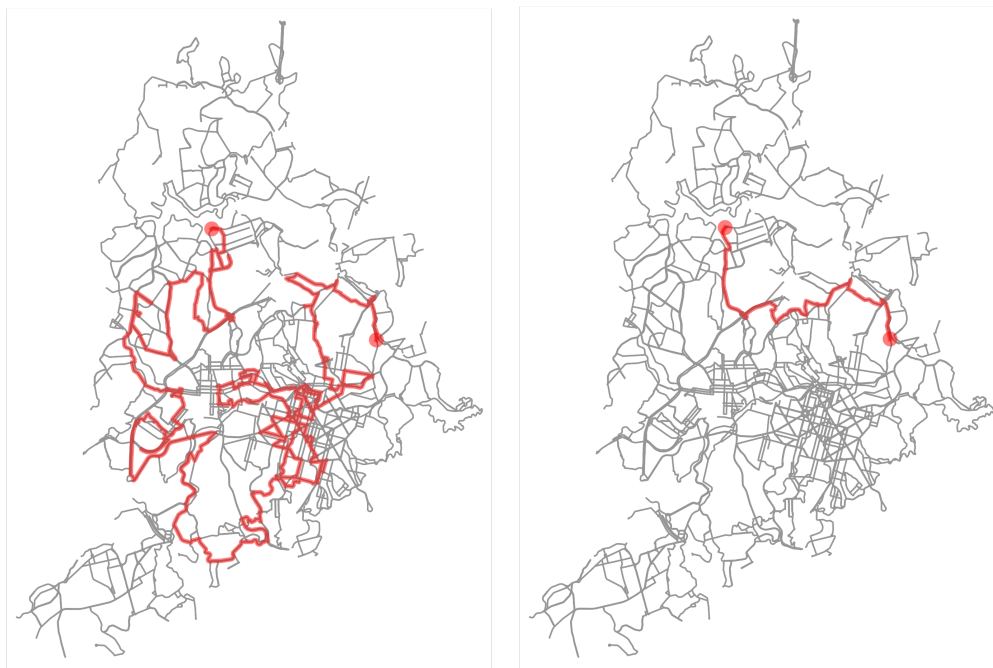
Rota com nx.shortest_path: [29095351, 9048339266, 303343655, 1494808236, 1494808222, 6429380808]
Quantidade de nós: 51

Rota com DFS: [29095351, 8951975509, 9048339273, 8996493468, 8952002417, 8952002418, 6370080808]
Quantidade de nós: 730
```

Figura 3 - Saída apresentada para comparação dos algoritmos

A partir dessa breve demonstração foi possível recordar conceitos e inferir que o resultado dos algoritmos Dijkstra e A* será idêntico quando ambos usam o mesmo grafo e o mesmo peso nas arestas e a heurística do A* é admissível e consistente (tal qual a distância euclidiana). Assim, A* encontra o mesmo caminho ótimo que Dijkstra, podendo apenas ter uma quantidade de nós explorados mais eficiente mas resultando na mesma rota final.

Dada a grande diferença entre os algoritmos DFS e A*, foi salvo e impresso na tela uma imagem mostrando o caminho escolhido por cada uma das estratégias, apresentadas abaixo nas Figuras 4 e 5.



Figuras 4 e 5 - Rotas traçadas com DFS e A*, respectivamente.

5. Conclusão

A partir desta atividade prática, para a resolução do problema proposto, pude implementar um agente de busca que informa rotas entre museus de Belo Horizonte, além de reforçar conceitos de programação em python e boas práticas. Pode-se perceber através deste trabalho como fazer um agente, utilizar um grande modelo de linguagem local e comparar algoritmos de busca em grafos, com a utilização de bibliotecas auxiliares para consolidar o resultado esperado.

Além disso, foi um ótimo exercício de revisão juntamente com o aprendizado do novo conteúdo apresentado pela disciplina de Introdução à Inteligência Artificial e durante sua modelagem e execução pude perceber minhas maiores dificuldades: decisões de projeto antes da implementação e integração dos algoritmos de busca com o modelo a ser utilizado pelo agente.

Referências Bibliográficas

- OPENSTREETMAP.
Tag:tourism=museum. Disponível em:
<https://wiki.openstreetmap.org/wiki/Tag%3Atourism%3Dmuseum>. Acesso em: 02 mar. 2025.
- HUGGING FACE. *smolagents documentation*. Disponível em:
<https://huggingface.co/docs/smolagents/in dex>. Acesso em: 05 mar. 2024.
- HUGGING FACE. *Introduction to Agents*. Disponível em:
<https://huggingface.co/learn/agents-course/unit1/introduction>. Acesso em: 15 mar. 2025.
- CHAIMOWICZ, Luiz; PAPPA, Gisele. Slides virtuais da disciplina de Introdução à Inteligência Artificial. Disponibilizado via Moodle. Departamento de Ciência da Computação, Universidade Federal de Minas Gerais, Belo Horizonte.
- GOSLING, Marlusa de Sevilha; PEREIRA, Gisele de Araujo; VERA, Luciana Alves Rodas; COELHO, Mariana de Freitas; LIMA, Carlos Gabriel de Azevedo. *Vamos fazer algo diferente? Um estudo exploratório sobre motivações de visita  o a museus*. Viann.a Sapiens, Juiz de Fora, v. 5, n. 2, p. 336–360, 2014.
- Dispon  vel em:
<https://www.academia.edu/14533761/>. Acesso em: 16 maio 2025.
- MORAIS, Leonardo. *CCBB de Belo Horizonte est   entre os museus mais visitados do mundo; veja ranking*. Di  rio do Com  rcio, Belo Horizonte, 3 abr. 2024. Dispon  vel em:
<https://diariodocomercio.com.br/variedade s/museus-belo-horizonte-mais-visitados-m undo/>. Acesso em: 16 maio 2025.
- VALADARES, Carolina. *Pesquisa aponta melhores museus do Brasil*. Minist  rio do Turismo, 13 out. 2015. Dispon  vel em:
<https://www.gov.br/turismo/pt-br/assuntos/noticias/pesquisa-aponta-melhores-museus -do-brasil>. Acesso em: 16 maio 2025.
- BOEING, Geoff. *OSMnx 2.0.3 Documentation*. Dispon  vel em:
<https://osmnx.readthedocs.io/en/stable/>. Acesso em: 15 maio 2025.
- NETWORKX. *Shortest Paths — NetworkX 3.4.2 documentation*. Dispon  vel em:
https://networkx.org/documentation/stable/reference/algorithms/shortest_paths.html. Acesso em: 15 maio 2025.

AP  NDICE A - Dificuldades pessoais

Durante a realiza  o deste trabalho pr  tico diversos problemas atrapalharam o desenvolvimento de um melhor projeto, entre eles a execu  o de LLM local e erro no servidor utilizado por “`ox.graph.graph_from_place()`”

Ao buscar op   es de como rodar o agente diversas possibilidades foram encontradas, entre elas utilizar um modelo com tokens limitados e/ou pago se mostrou incoerente com o escopo do trabalho. Dessa forma, seguindo as recomenda  es do curso sugerido de Agentes do Hugging Face foi baixado e instalado o ollama/qwen2:7b. Ap  s todo o processo, ao tentar

executar mesmo o programa mais simples, como o dado de exemplo no vídeo disponibilizado no moodle e sem usar nenhuma função de busca, os programas demoravam muito para responder (Figura 1, tomando aproximadamente 5 minutos para rodar o mesmo código que no vídeo levou 19,07 seg) e então constatou-se que estava sendo executado na CPU (Figuras 2 e 3) e não na GPU (Radeon 520), visto que a mesma era incompatível com o Ollama.

```
[11] ✓ 4m 58.1s
agent = CodeAgent(tools=[FindRouteTool()], model=model, add_base_tools=False)
agent.run("How can I go from Margô Drinkeria to Serilla Cozinha & Drinks?")
```

Figura 1 - Tempo de execução de um código sem busca apresentado como base

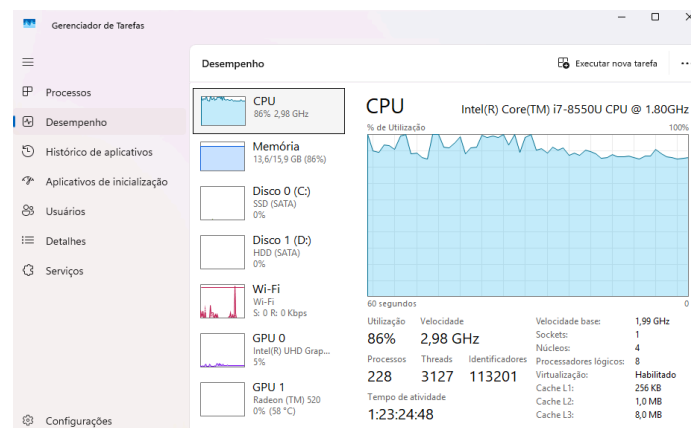


Figura 2 - Uso da CPU durante a execução

Processos		Executar nova tarefa				Finalizar tarefa	..
Nome	Status	100% CPU	87% Memória	0% Disco	0% Rede		
> ollama (2)		84,8%	5.448,9 MB	0 MB/s	0 Mbps		
> Visual Studio Code (18)		4,5%	1.660,3 MB	0 MB/s	0 Mbps		

Figura 3 - Consumo da CPU por processos durante a execução

Dessa forma, para evitar problemas foi escolhido mudar o hardware que estava sendo executado, usando o notebook de um amigo com GPU compatível (RTX 4060) e versionamento sendo feito via Git e GitHub.

Ademais, na primeira execução do código um erro foi um empecilho: “Erro no servidor (“ConnectTimeout: HTTPConnectionPool(host='overpass-api.de', port=443): Max retries exceeded with url: /api/interpreter” na execução de G = ox.graph.graph_from_place(...))”, sendo necessário a mudança para salvar o grafo em um arquivo (bh.graphml) e o envio do mesmo.

Por fim, durante todo o desenvolvimento foram pensadas estratégias possíveis de melhoria e recursos futuros, não implementados para melhor clareza do código entregue. Todas essas modificações propostas estão devidamente comentadas nas células de texto, como, por exemplo, a alternativa de solicitar ao usuário a entrada com “input()”.

Código e arquivos auxiliares disponíveis em: <https://github.com/carlabferreira/SearchAgent>.